

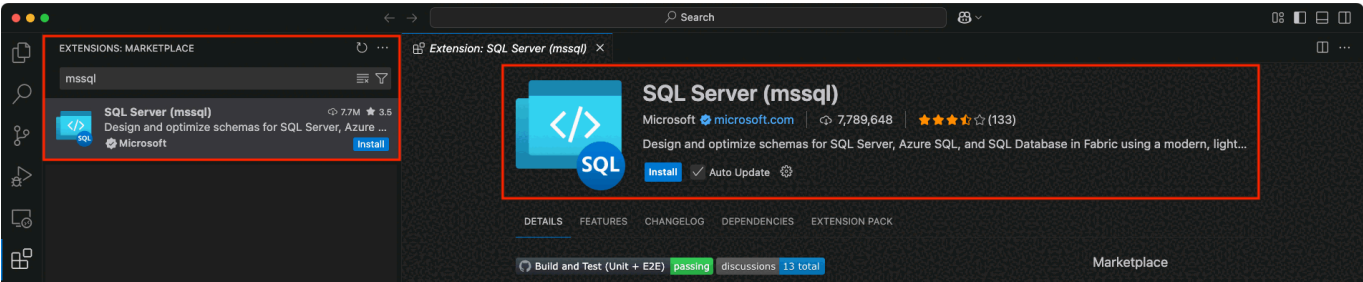
Material Complementar — Recursos Visuais e Low-Code do VS Code com a Extensão MSSQL

Curso de Integridade Fiscal — Banco de Dados **curso_integridade_fiscal** Complementa os módulos de BULK INSERT, ALTER TABLE e o exercício integrador

1. Objetivo

Todos os módulos anteriores do curso foram construídos em torno da escrita direta de comandos T-SQL (**BULK INSERT**, **CREATE TABLE**, **ALTER TABLE**, **UPDATE**). Esse caminho é essencial para entender o que acontece "por baixo do capô" — mas no dia a dia da auditoria fiscal, muitas dessas tarefas podem ser resolvidas mais rapidamente por interfaces gráficas, sem abrir mão da precisão técnica.

Este material apresenta os recursos **visuais e low-code** da extensão MSSQL para VS Code: assistentes de importação, editores de dados em grade (estilo planilha), e designers de tabela e de esquema com diagramação e arrastar-e-soltar. A ideia não é substituir o T-SQL — é saber quando cada abordagem é mais eficiente.



A extensão MSSQL instalada e ativa no VS Code — o ícone na Activity Bar e a vista de Conexões confirmam a instalação. Fonte: Microsoft Learn.

2. Panorama dos recursos

A extensão MSSQL para Visual Studio Code fornece ferramentas para conectar a bancos de dados, gerenciar e desenhar esquemas, explorar objetos do banco de dados, executar consultas T-SQL e visualizar planos de execução. Os recursos relevantes para um fluxo de trabalho **low-code** são:

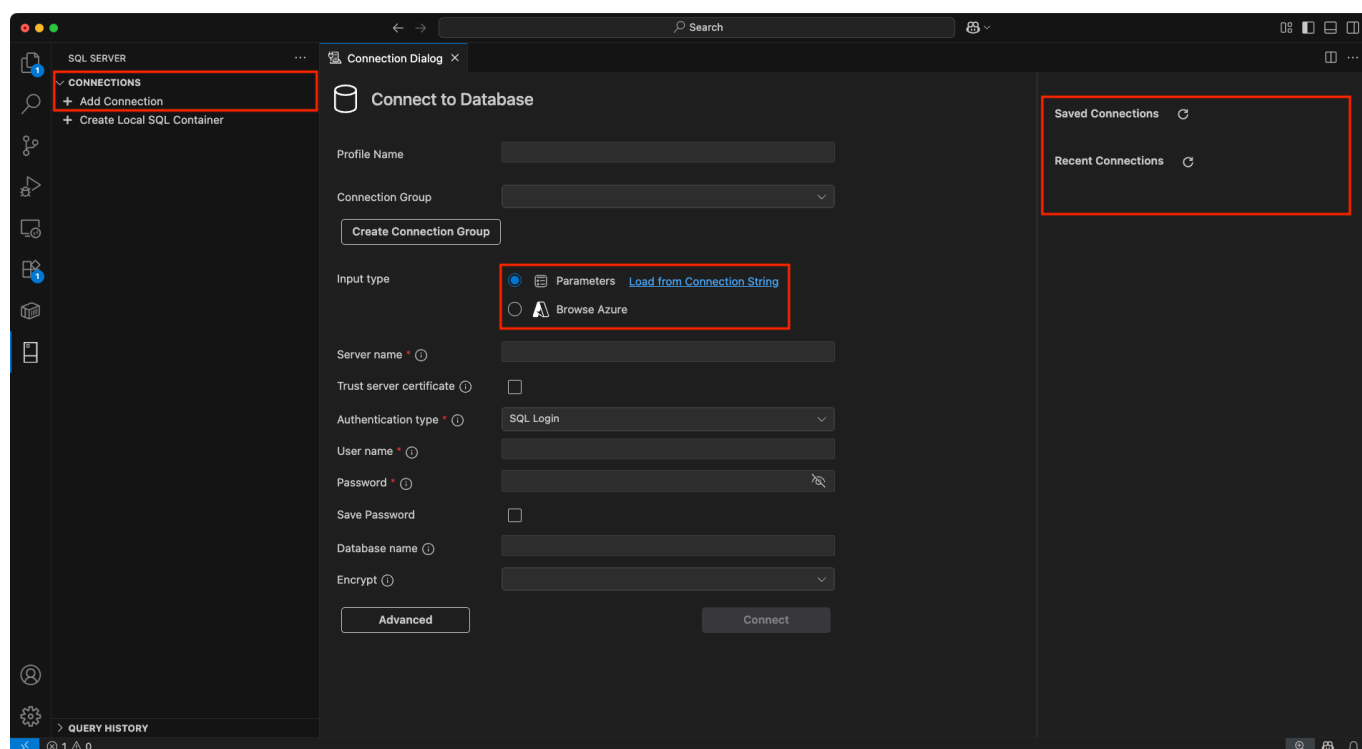
Recurso	O que substitui em T-SQL	Situação
Importar Arquivo Plano (Flat File Import)	BULK INSERT	Disponibilidade geral
Table Designer	CREATE TABLE / ALTER TABLE	Disponibilidade geral
Schema Designer	CREATE TABLE com FOREIGN KEY , diagramação de modelo	Disponibilidade geral

Recurso	O que substitui em T-SQL	Situação
Visualizar & Editar Dados	INSERT / UPDATE / DELETE simples	Disponibilidade geral
Explorador de Objetos com filtros	Consultas em sys.tables , sys.columns	Disponibilidade geral
Schema Designer com GitHub Copilot	Modelagem inteira a partir de linguagem natural	Disponibilidade geral

3. Importação de CSV sem escrever BULK INSERT

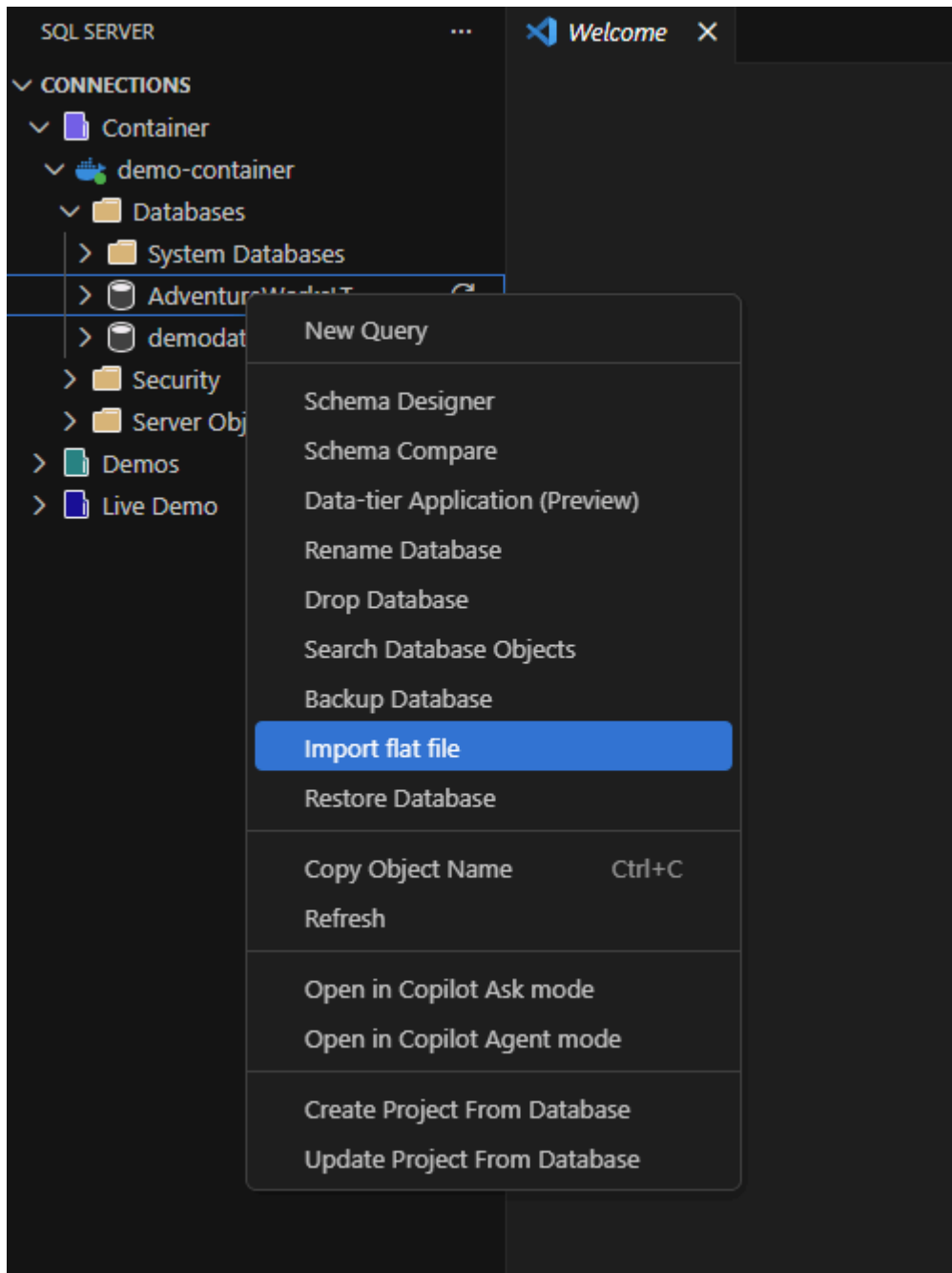
O assistente **Importar Arquivo Plano** (*Flat File Import*) é o equivalente visual direto do que foi feito manualmente no módulo de BULK INSERT.

Antes da importação, a conexão com o banco é feita pela caixa de diálogo de conexão da extensão:



Tela de conexão — permite conectar por parâmetros, string de conexão ou navegando pelo Azure. Fonte: Microsoft Learn.

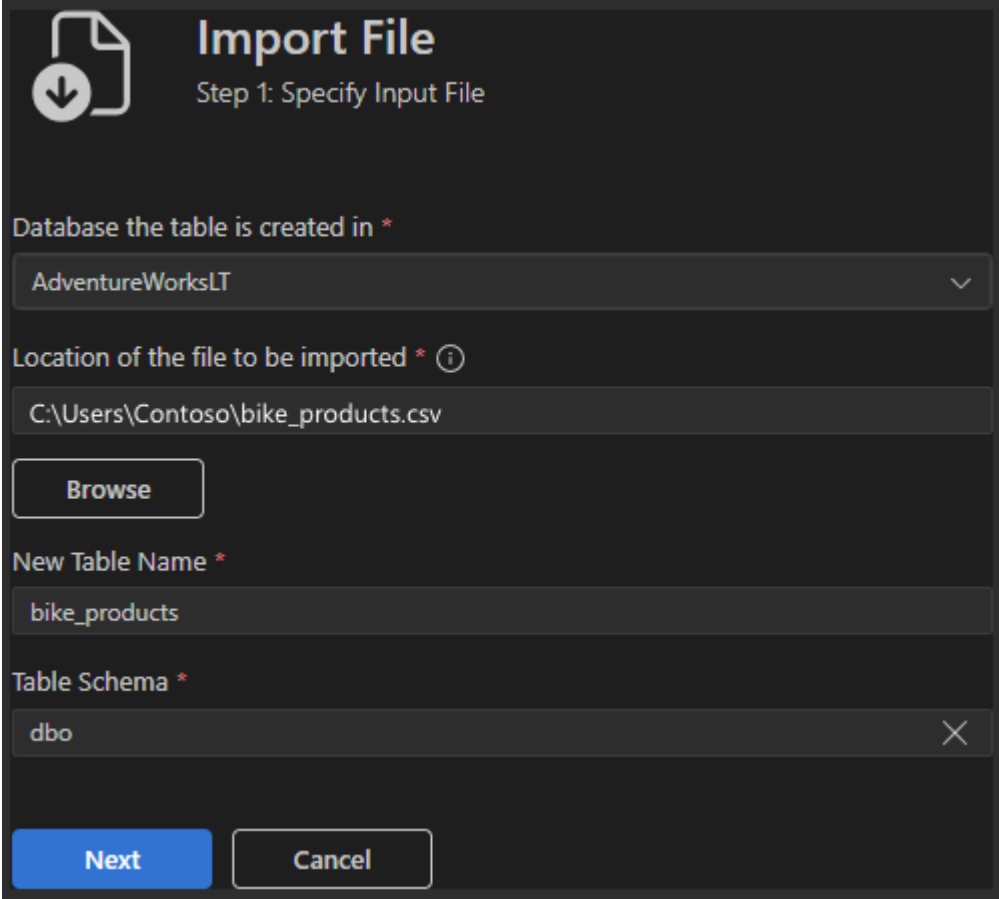
Como acessar o assistente: no Object Explorer (vista de Conexões), expanda a conexão e o nó **Databases**, clique com o botão direito no banco de dados de destino e selecione **Import flat file**.



Ponto de entrada do assistente — botão direito no banco de dados → *Import flat file*. Fonte: Microsoft Learn.

O assistente conduz o usuário por etapas:

Passo 1 — Especificar o arquivo de entrada: informe o banco de destino, o caminho do arquivo `.csv/.txt`, o nome da nova tabela e o schema (ex.: `dbo`).



Import File
Step 1: Specify Input File

Database the table is created in *

AdventureWorksLT

Location of the file to be imported * ⓘ

C:\Users\Contoso\bike_products.csv

Browse

New Table Name *

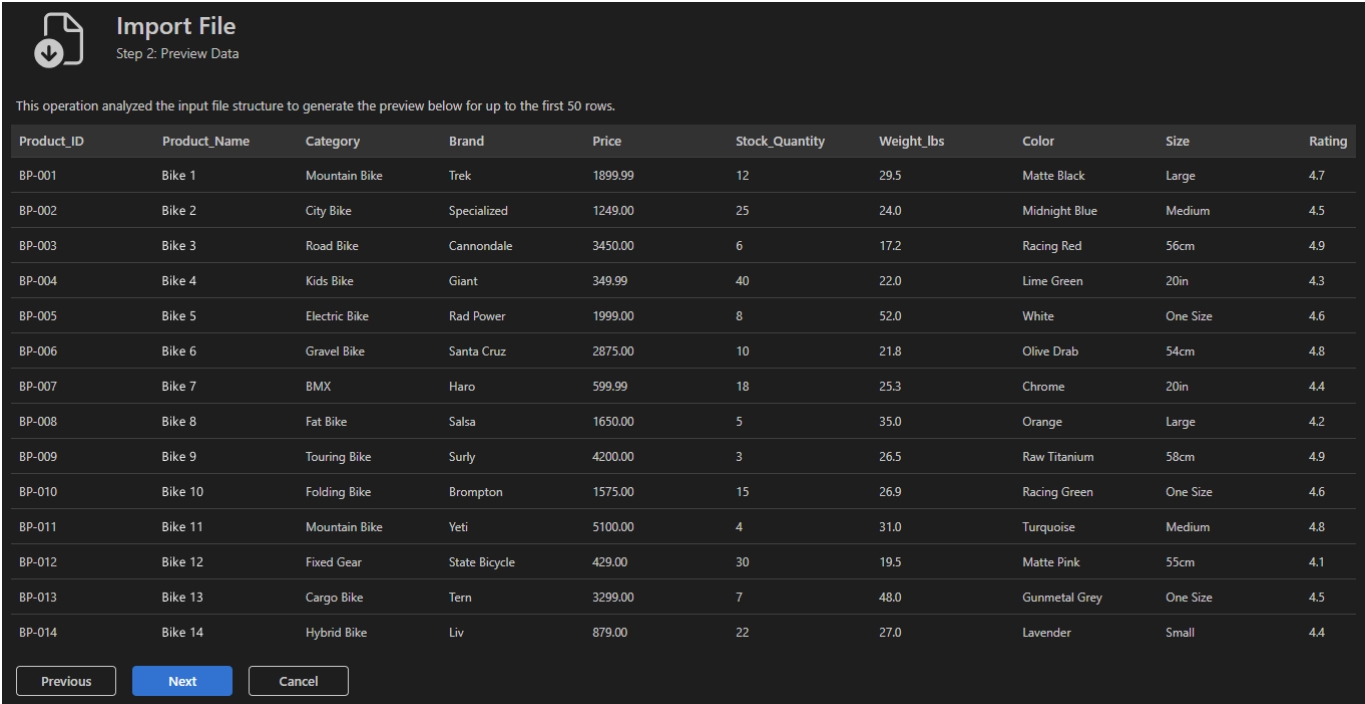
bike_products

Table Schema *

dbo

Next Cancel

Passo 2 — Pré-visualizar os dados: a extensão detecta automaticamente os nomes de colunas e os tipos de dados, poupando a etapa manual de decidir se cada coluna é **VARCHAR**, **INT**, **DATE**, etc. Esta tela permite validar o alinhamento das colunas e o delimitador usado antes de prosseguir.



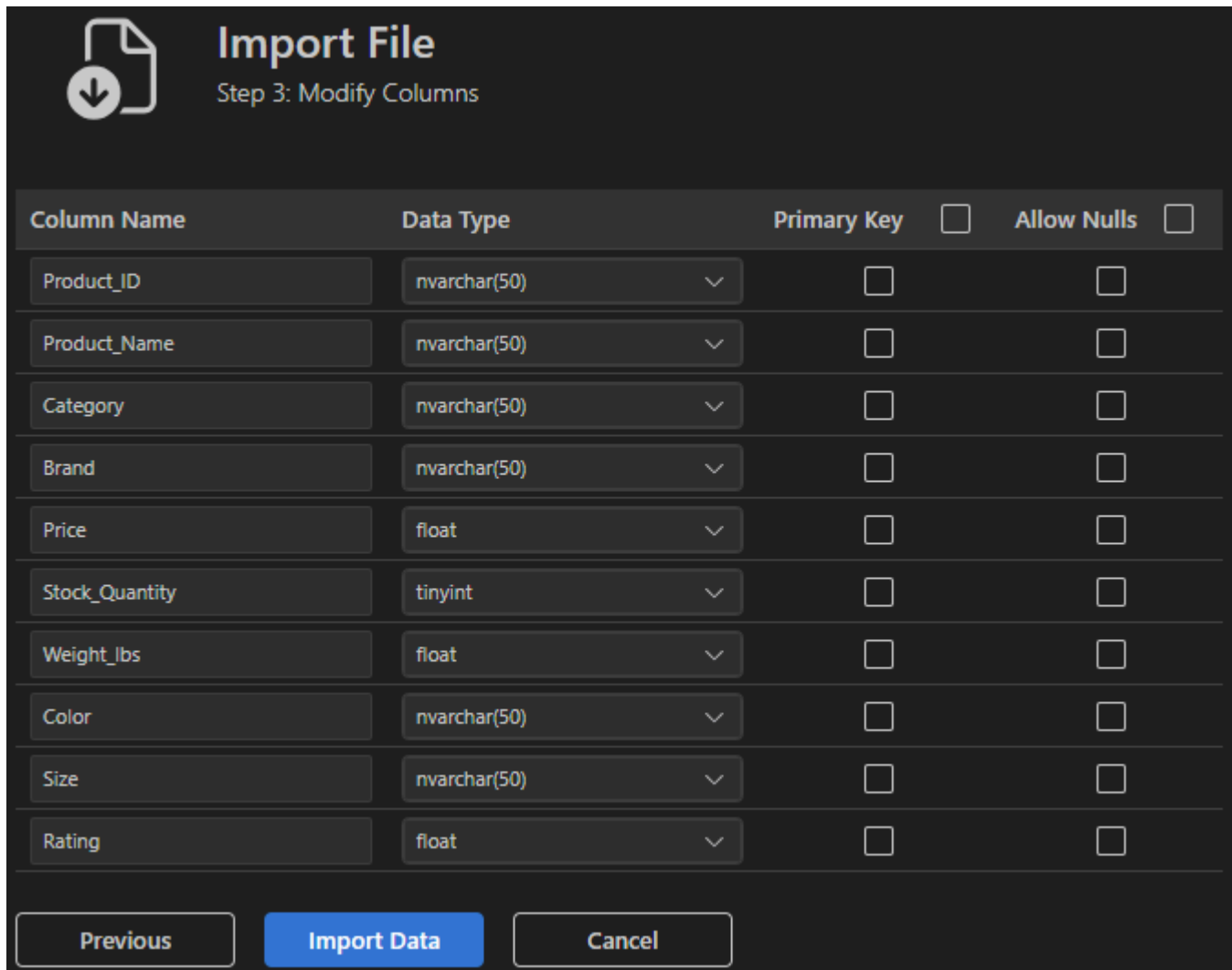
Import File
Step 2: Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

Product_ID	Product_Name	Category	Brand	Price	Stock_Quantity	Weight_lbs	Color	Size	Rating
BP-001	Bike 1	Mountain Bike	Trek	1899.99	12	29.5	Matte Black	Large	4.7
BP-002	Bike 2	City Bike	Specialized	1249.00	25	24.0	Midnight Blue	Medium	4.5
BP-003	Bike 3	Road Bike	Cannondale	3450.00	6	17.2	Racing Red	56cm	4.9
BP-004	Bike 4	Kids Bike	Giant	349.99	40	22.0	Lime Green	20in	4.3
BP-005	Bike 5	Electric Bike	Rad Power	1999.00	8	52.0	White	One Size	4.6
BP-006	Bike 6	Gravel Bike	Santa Cruz	2875.00	10	21.8	Olive Drab	54cm	4.8
BP-007	Bike 7	BMX	Haro	599.99	18	25.3	Chrome	20in	4.4
BP-008	Bike 8	Fat Bike	Salsa	1650.00	5	35.0	Orange	Large	4.2
BP-009	Bike 9	Touring Bike	Surly	4200.00	3	26.5	Raw Titanium	58cm	4.9
BP-010	Bike 10	Folding Bike	Brompton	1575.00	15	26.9	Racing Green	One Size	4.6
BP-011	Bike 11	Mountain Bike	Yeti	5100.00	4	31.0	Turquoise	Medium	4.8
BP-012	Bike 12	Fixed Gear	State Bicycle	429.00	30	19.5	Matte Pink	55cm	4.1
BP-013	Bike 13	Cargo Bike	Tern	3299.00	7	48.0	Gunmetal Grey	One Size	4.5
BP-014	Bike 14	Hybrid Bike	Liv	879.00	22	27.0	Lavender	Small	4.4

Previous Next Cancel

Passo 3 — Modificar colunas: é possível ajustar nomes de colunas, tipos de dados, chaves primárias e nulidade antes de confirmar.



Import File
Step 3: Modify Columns

Column Name	Data Type	Primary Key	Allow Nulls
Product_ID	nvarchar(50)	☐	☐
Product_Name	nvarchar(50)	☐	☐
Category	nvarchar(50)	☐	☐
Brand	nvarchar(50)	☐	☐
Price	float	☐	☐
Stock_Quantity	tinyint	☐	☐
Weight_lbs	float	☐	☐
Color	nvarchar(50)	☐	☐
Size	nvarchar(50)	☐	☐
Rating	float	☐	☐

Previous Import Data Cancel

Ao selecionar **Import Data**: a tabela é criada e os dados são importados em um único fluxo contínuo, sem exigir um **CREATE TABLE** prévio nem a instrução **BULK INSERT**.

Fonte das capturas de tela desta seção: tutorial oficial "Database Operations — MSSQL Extension for Visual Studio Code" — Microsoft Learn.

Aplicação no cenário do curso: ao receber o arquivo `nfe_importacao_lote.csv` do exercício integrador, em vez de escrever manualmente o **CREATE TABLE stg_nfe_importacao** e depois o **BULK INSERT**, o auditor poderia:

- Clicar com o botão direito no banco `curso_integridade_fiscal` → **Import Flat File**;
- Apontar para o CSV e deixar o assistente inferir os tipos;
- Ajustar manualmente `valor_total` e `data_emissao` para **VARCHAR** (já que o arquivo contém erros propositais) — o mesmo raciocínio do módulo anterior, só que decidido visualmente, clicando no tipo da coluna em vez de escrevê-lo;
- Concluir o assistente, que cria a tabela e carrega as 40 linhas automaticamente.

Quando preferir o assistente em vez do **BULK INSERT manual:** cargas pontuais, exploratórias, ou quando o formato do arquivo ainda não é totalmente conhecido (o assistente ajuda a descobrir a estrutura). **Quando preferir o **BULK INSERT** escrito:** cargas repetitivas, agendadas, ou que precisam ser versionadas como script (ex.: parte de uma rotina de ETL documentada).

4. Table Designer — criando e alterando tabelas visualmente

O **Table Designer** oferece uma interface para o que os módulos de modelagem e de **ALTER TABLE** fizeram via código.

Como acessar: botão direito em **Tables** no Object Explorer → **New Table** (para criar) ou botão direito em uma tabela existente → **Design** (para alterar).

Principais capacidades da interface:

- **Colunas** — *adicionar novas colunas, definir tipos de dados, definir a anulabilidade e especificar valores padrão, além de designar uma coluna como chave primária ou coluna de identidade diretamente na interface;*
- **Chave primária** — *definir uma ou mais colunas como chave primária da tabela;*
- **Índices** — *criar e gerenciar índices para melhorar o desempenho das consultas;*
- **Chaves estrangeiras** — *definir relacionamentos entre tabelas referenciando chaves primárias de outras tabelas;*
- **Check constraints** — *configurar regras de validação (equivalente visual ao **CHECK** que foi usado no exercício integrador para `status_validacao`).*

Um diferencial importante: o designer mantém um painel **Script como Criar**, que mostra em tempo real o T-SQL equivalente a cada alteração feita visualmente. Isso significa que o material continua "auditável" — o auditor pode clicar para montar a tabela e, ao final, copiar ou revisar o script gerado antes de aplicá-lo, com a opção de **Publicar** (aplica direto no banco) ou **Copiar script** (para versionar ou ajustar manualmente).

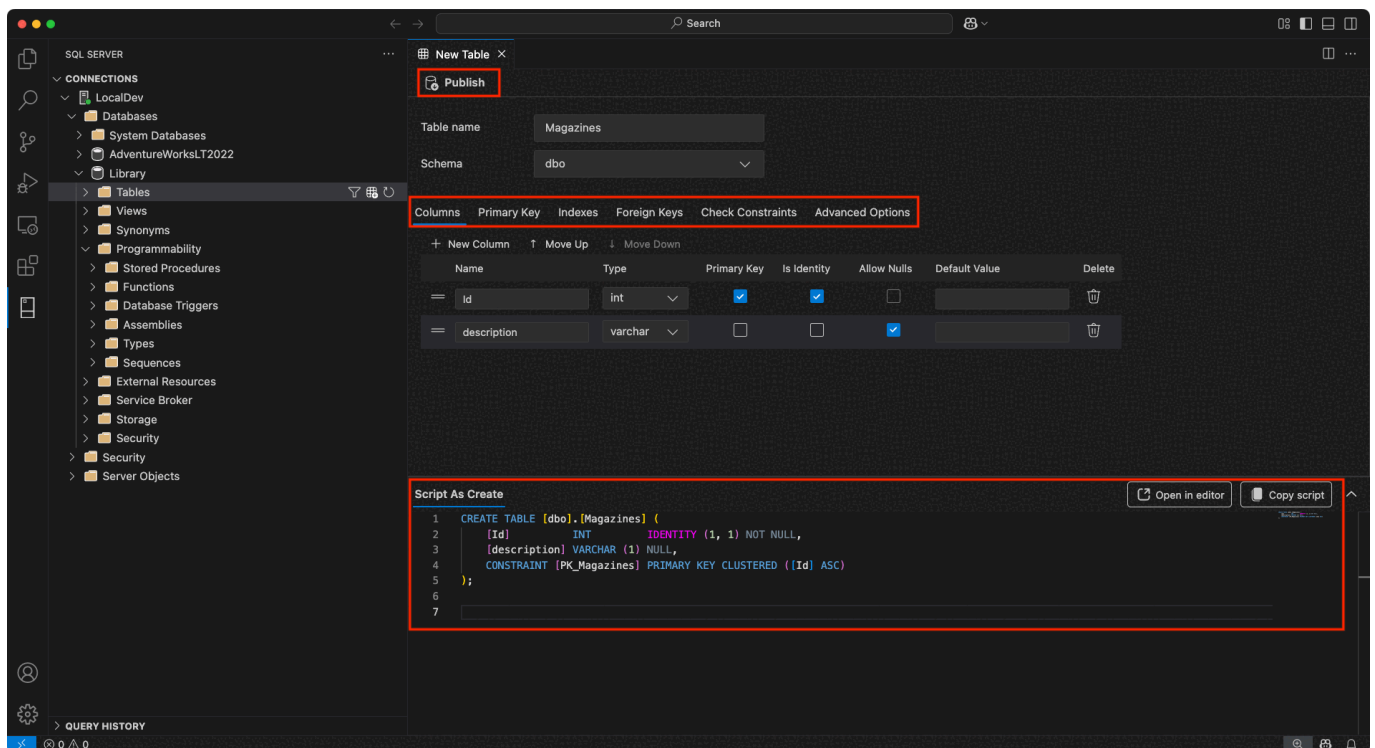


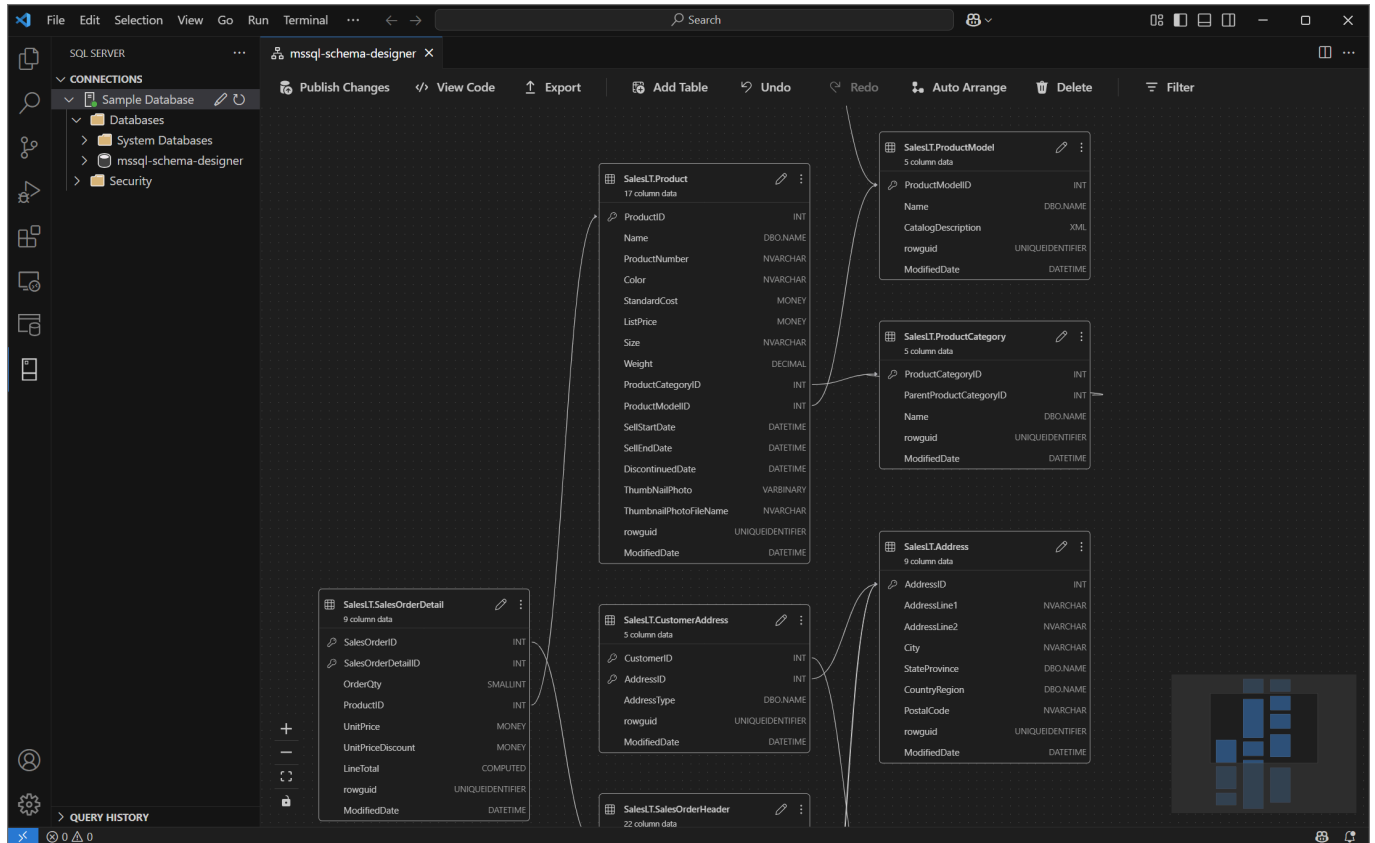
Table Designer aberto no VS Code — grade de colunas à esquerda e painel de script T-SQL gerado à direita.
Fonte: Microsoft Learn.

Aplicação no cenário do curso: as colunas de controle adicionadas no exercício integrador (`status_validacao`, `motivo_rejeicao`, `data_analise`, `auditor_responsavel`, e a constraint de **CHECK**) poderiam ter sido adicionadas inteiramente pelo Table Designer, sem escrever nenhum **ALTER TABLE** —

bastando abrir a tabela em modo de edição, clicar em "adicionar coluna" quatro vezes e configurar a constraint na aba correspondente.

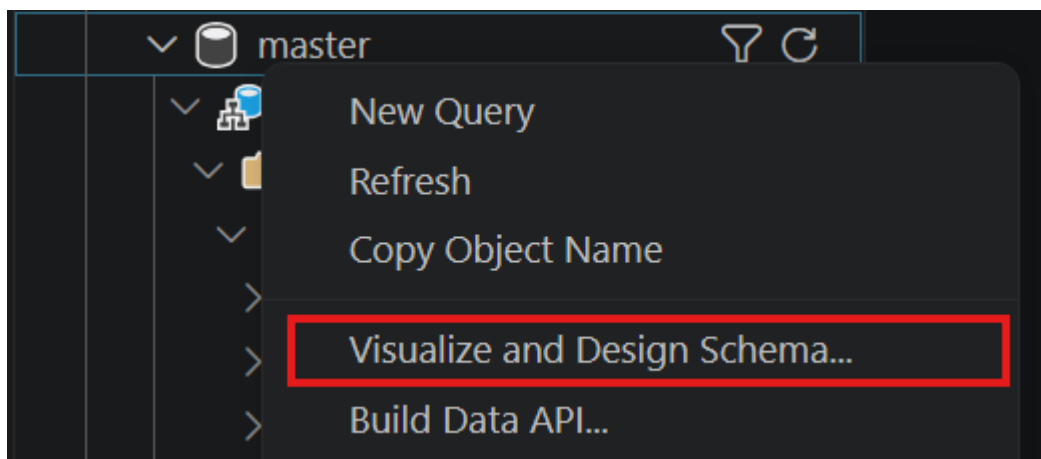
5. Schema Designer — modelagem visual com diagrama

O **Schema Designer** vai além de uma tabela isolada: ele apresenta o banco inteiro como um diagrama interativo, no estilo dos diagramas ER trabalhados nos módulos de modelagem relacional.



Visão geral do canvas do Schema Designer, com várias tabelas e seus relacionamentos exibidos como um diagrama. Fonte: Microsoft Learn.

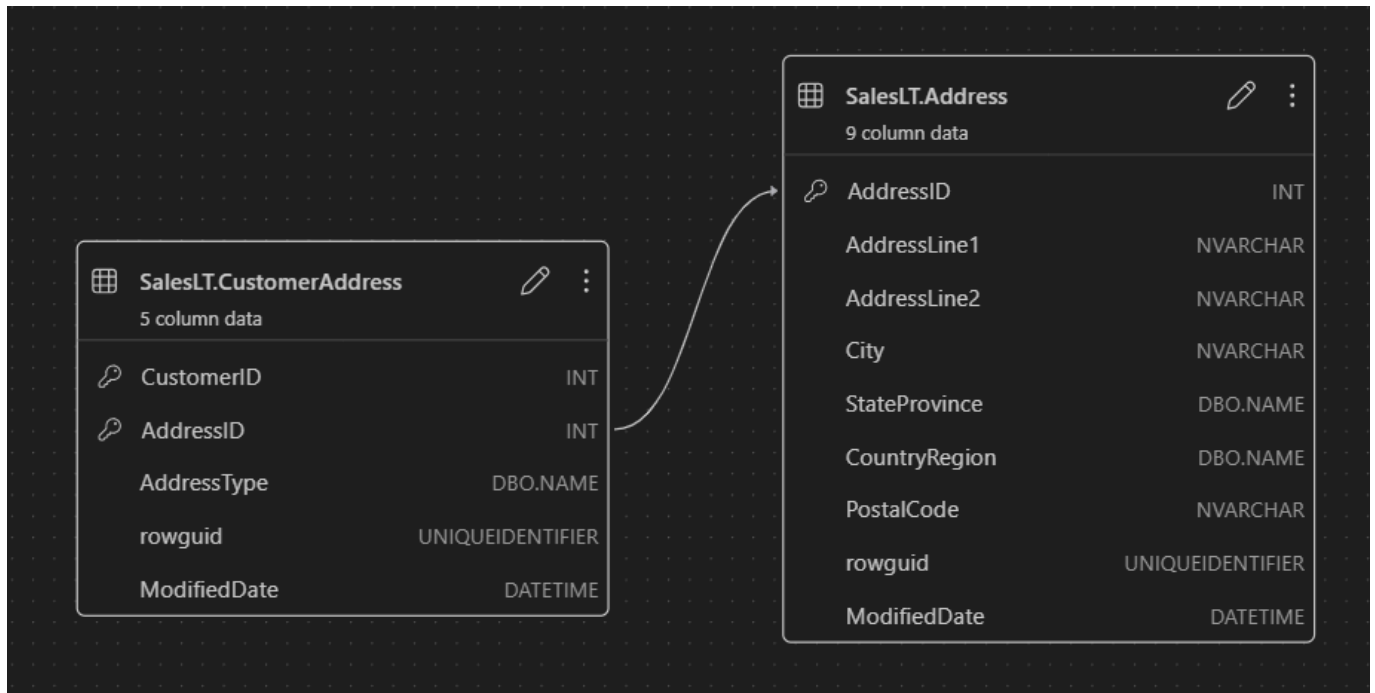
Como acessar: clique com o botão direito no banco de dados no Object Explorer e selecione **Visualize and Design Schema**.



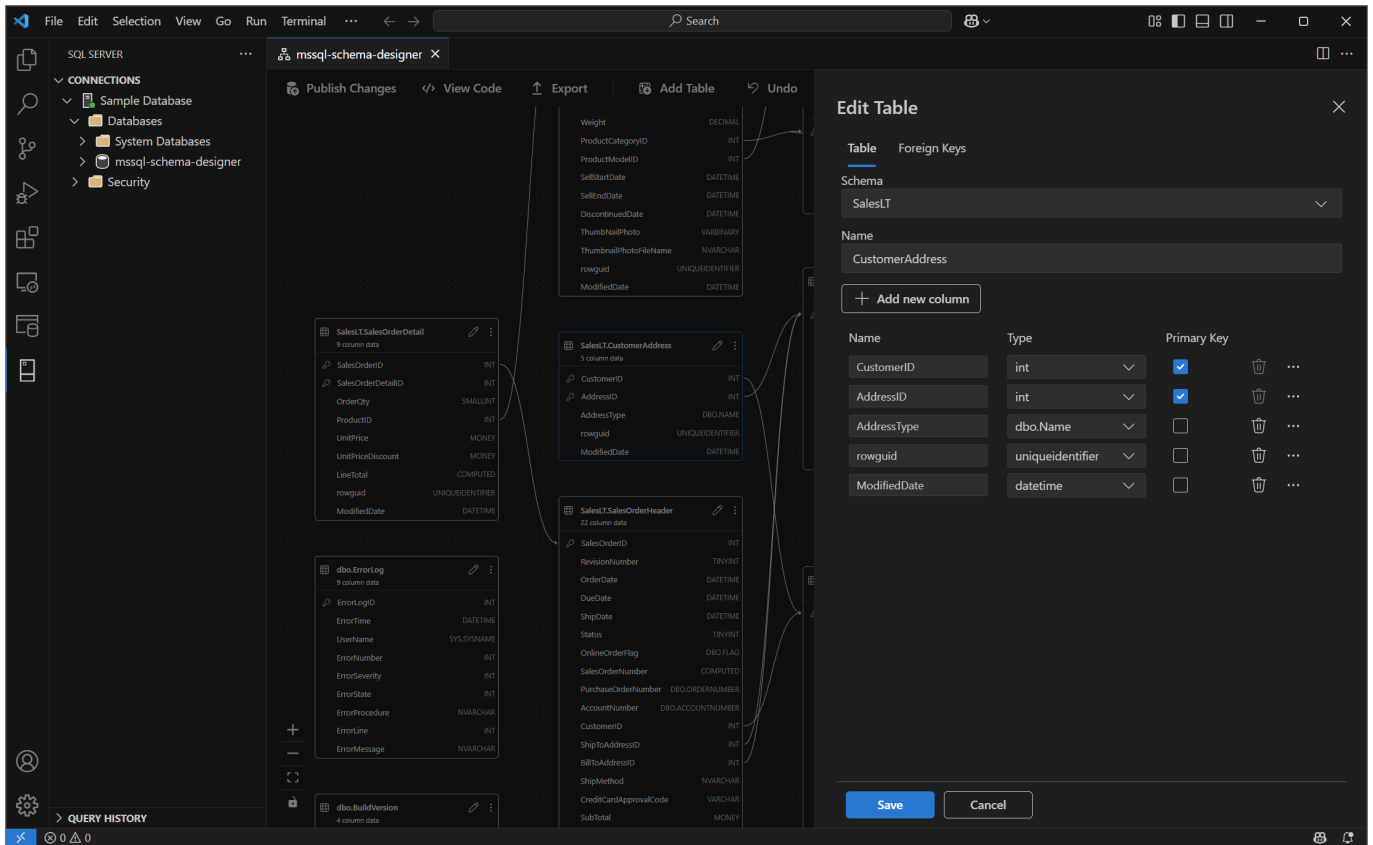
Recursos principais:

- Interface visual para criar e gerenciar múltiplas tabelas simultaneamente, com disposição automática das tabelas no diagrama;
- Arrastar e soltar para reposicionar tabelas e reorganizar visualmente o modelo, além de recursos de busca, filtro, zoom e um mini-mapa para diagramas grandes;
- Criação de relacionamentos (chaves estrangeiras) desenhando uma linha entre colunas de tabelas diferentes, em vez de escrever **FOREIGN KEY REFERENCES**;
- Geração automática do script T-SQL correspondente ao diagrama, para revisão antes de publicar.

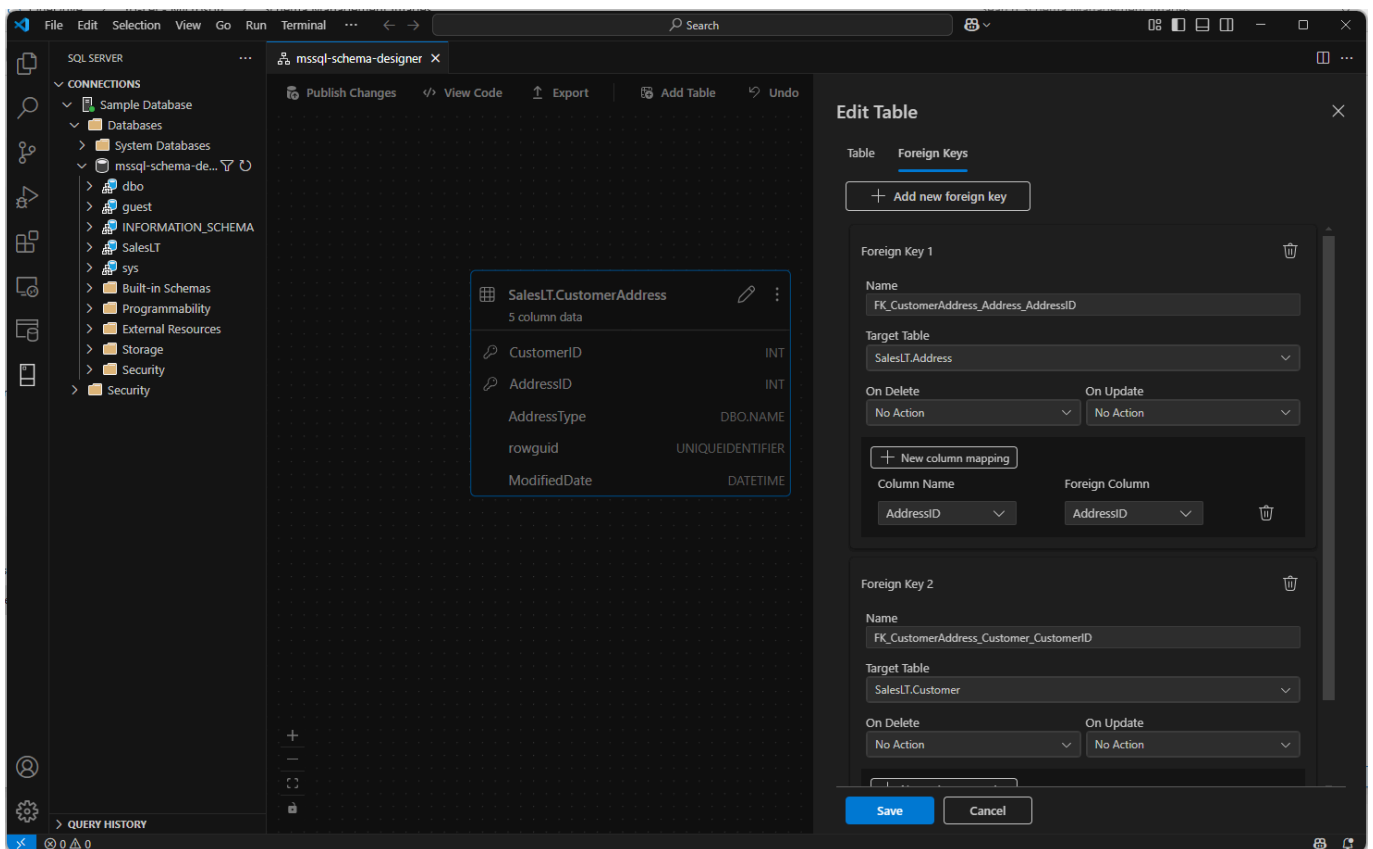
Cada tabela no diagrama exibe seu nome, colunas, tipos de dados e a chave primária (identificada por um ícone de chave):

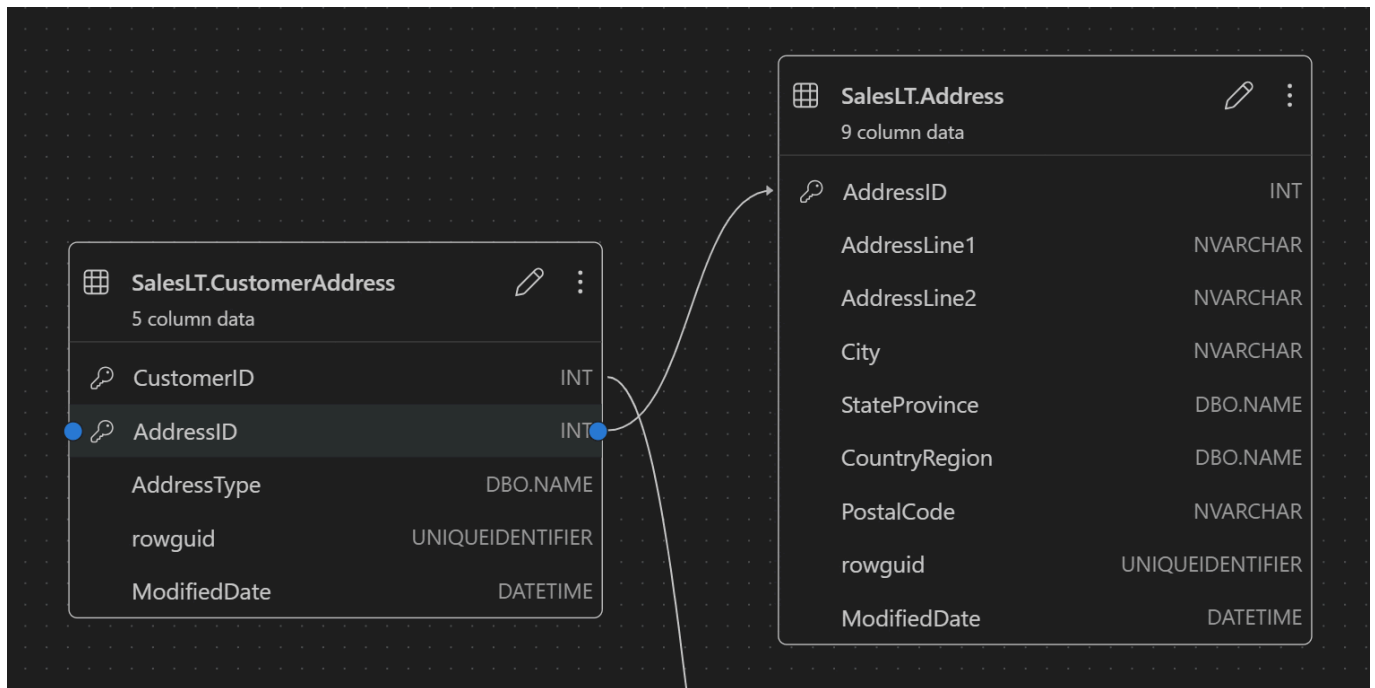


Para editar uma tabela, basta clicar no ícone de lápis, o que abre o painel **Table Editor** ao lado do canvas:

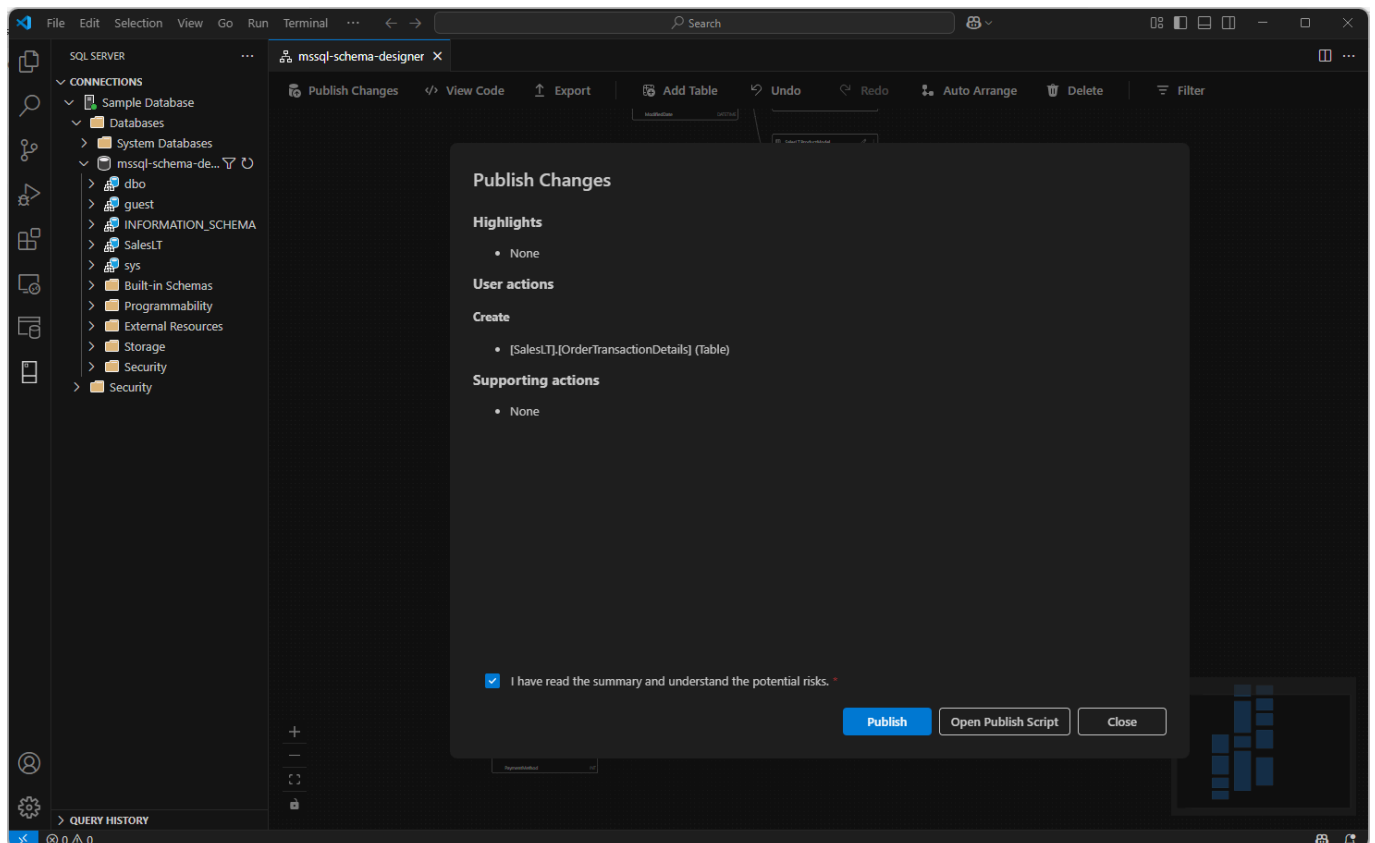


Os relacionamentos entre tabelas (chaves estrangeiras) podem ser gerenciados por um painel dedicado, ou desenhados diretamente arrastando uma seta de uma coluna a outra no diagrama:





Ao final da modelagem, o botão **Publish Changes** mostra um resumo de todas as alterações pendentes antes de aplicá-las ao banco:



Fonte das capturas de tela desta seção: tutorial oficial "Schema Designer — MSSQL Extension for Visual Studio Code" — Microsoft Learn.

Recurso avançado — Schema Designer com GitHub Copilot: a integração com o GitHub Copilot permite desenhar e evoluir esquemas de banco de dados usando linguagem natural diretamente no canvas visual. Em vez de adicionar tabelas, colunas e relacionamentos manualmente pela interface, é possível descrever o que se precisa em português (ou inglês) e o Copilot gera as tabelas, colunas, tipos de dados e relacionamentos automaticamente, refletidos em tempo real tanto no diagrama visual quanto no script T-SQL.

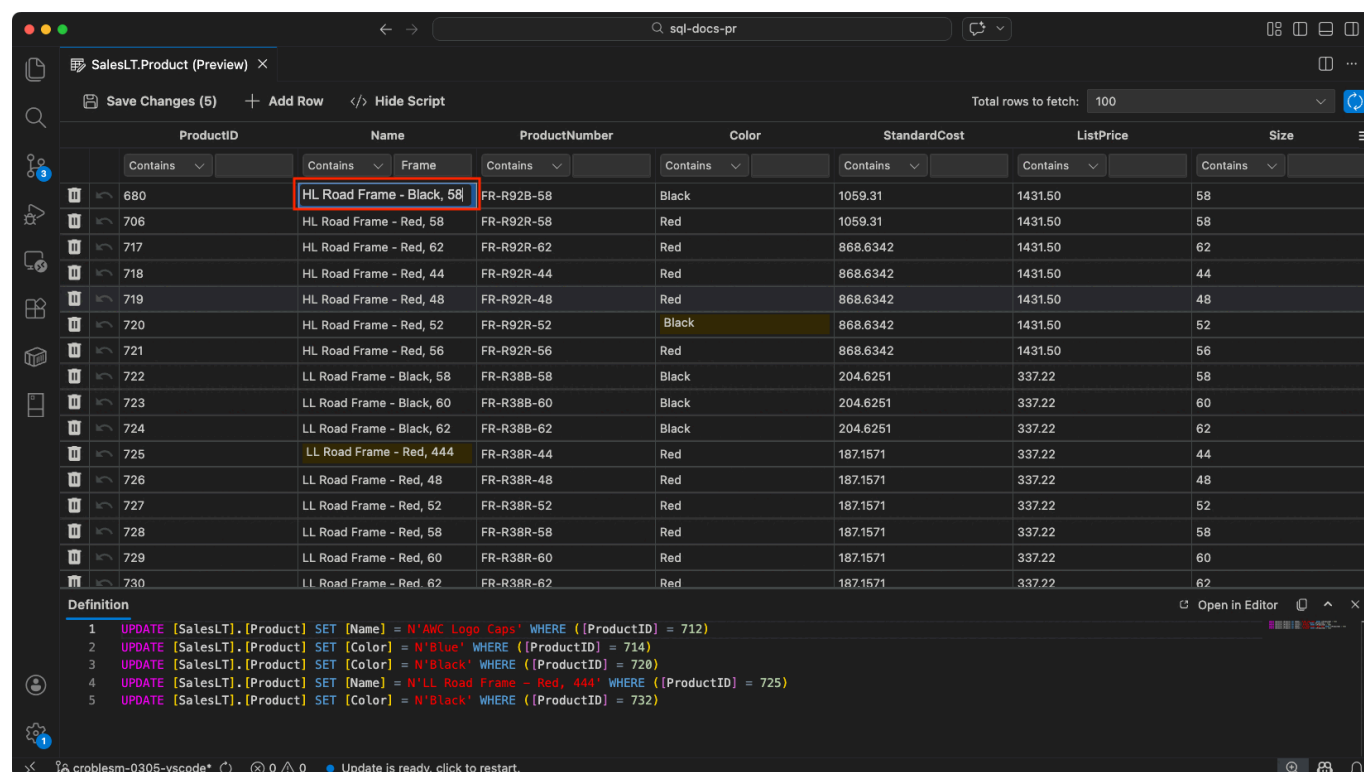
Aplicação no cenário do curso: o conjunto de entidades reutilizado em todos os módulos (**contribuinte**, **nfe**, **auto_infracao**, **auditor_fiscal**) poderia ser recriado do zero desenhando as quatro tabelas no canvas do Schema Designer, arrastando uma linha de **nfe.cnpj_emitente** até **contribuinte.cnpj** para materializar visualmente a chave estrangeira — o mesmo relacionamento que, em T-SQL, exigiria escrever a cláusula **FOREIGN KEY ... REFERENCES**.

6. Visualizar & Editar Dados — correção de dados sem UPDATE

Este é o recurso mais diretamente comparável ao módulo de **UPDATE/DELETE**.

Como acessar: botão direito em uma tabela no Object Explorer → **Edit Data** (também acessível dando um duplo clique na tabela).

Os dados da tabela abrem em uma grade dentro de uma aba do editor, em um layout semelhante a uma planilha, com navegação por páginas. Capacidades principais:



Grade de edição de dados aberta em uma aba do editor — layout semelhante a uma planilha, com paginação e o painel de script correspondente. Fonte: Microsoft Learn.

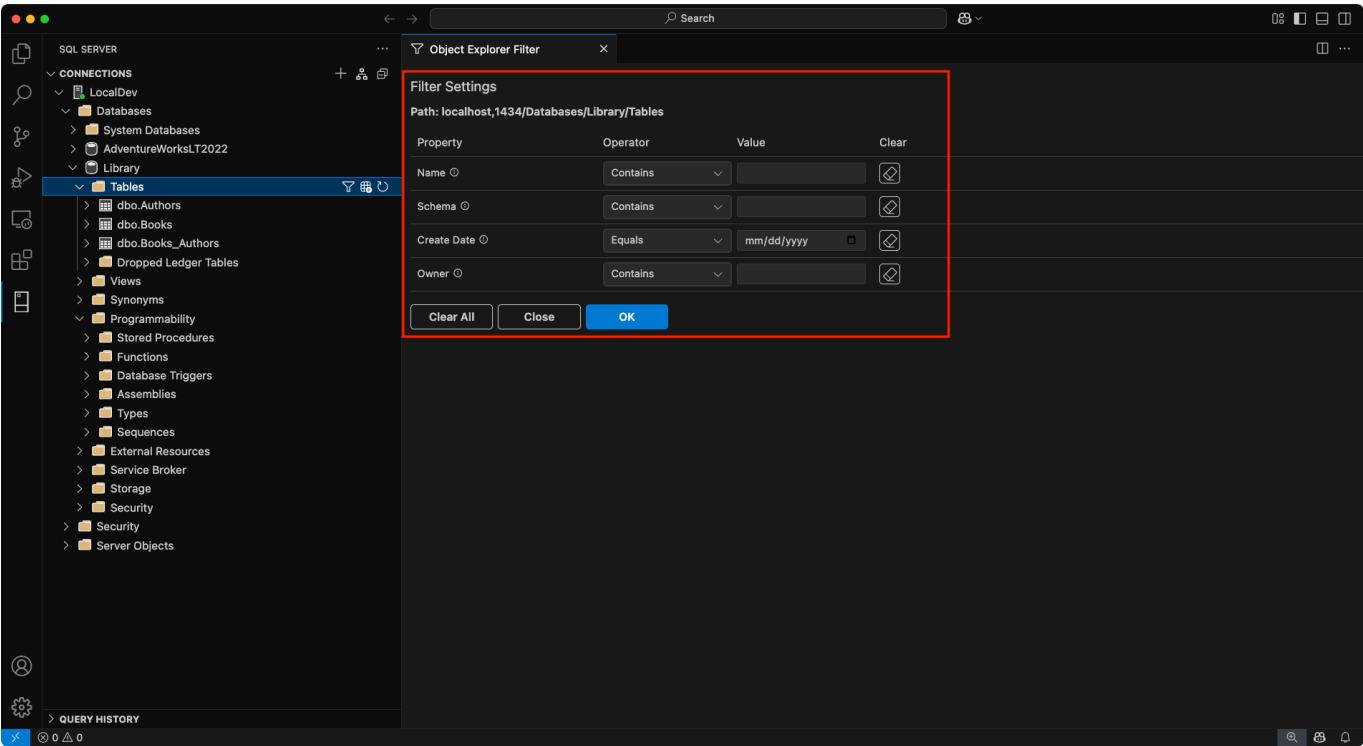
- **Edição em linha** — atualizar valores de células diretamente na grade, com validação em tempo real que sinaliza em vermelho entradas inválidas, como tipos de dados incorretos ou violações de constraint;
- **Adicionar e excluir linhas** — inserir ou remover registros sem escrever **INSERT/DELETE**;
- **Paginação** — navegar por grandes volumes de dados sem carregar tudo de uma vez;
- **Salvar alterações** — as edições ficam pendentes até que o usuário selecione "Salvar Alterações", mantendo controle total sobre quando as atualizações são gravadas no banco;
- **Mostrar script** — um painel apresenta, em modo somente leitura, o script DML equivalente a todas as ações feitas na grade — ou seja, mesmo corrigindo dados visualmente, é possível auditar exatamente qual **UPDATE/INSERT/DELETE** foi gerado.

Importante para o contexto de auditoria fiscal: essa "prestação de contas" via **Mostrar Script** é o que torna o recurso apropriado mesmo em um ambiente que exige rastreabilidade — nenhuma alteração acontece "invisível"; o T-SQL equivalente sempre pode ser conferido antes ou depois de salvar.

Limitação a ter em mente: correções em massa baseadas em regra de negócio (como os itens 7.1 a 7.7 do exercício integrador, que aplicam uma condição a dezenas de linhas de uma vez) continuam sendo mais eficientes em T-SQL puro. O editor em grade é ideal para correções pontuais, linha a linha — não para regras de triagem aplicadas a um lote inteiro.

7. Explorador de Objetos com filtros

Para localizar rapidamente tabelas, views e outros objetos em um banco com muitos elementos — situação comum à medida que o curso acumula módulos sobre o mesmo `curso_integridade_fiscal` — o *Object Explorer* permite aplicar filtros por propriedades como nome, proprietário ou data de criação, em múltiplos níveis da hierarquia do banco. É a alternativa visual a escrever consultas em `sys.tables` ou `sys.columns` para localizar objetos.



Filtros aplicados ao Object Explorer para restringir a lista de objetos exibidos. Fonte: Microsoft Learn.

8. Quando usar cada abordagem

Situação	Abordagem recomendada
Carga exploratória de um CSV novo, formato ainda incerto	Import Flat File (assistente)
Carga repetitiva, agendada, documentada como script	BULK INSERT escrito
Criar uma tabela nova rapidamente	Table Designer

Situação	Abordagem recomendada
Desenhar o modelo relacional completo de um novo módulo	Schema Designer
Corrigir 1 a 3 registros específicos	Visualizar & Editar Dados
Corrigir dezenas/centenas de registros por regra de negócio	UPDATE com WHERE
Entender o que uma constraint realmente faz	Escrever o ALTER TABLE ... CHECK manualmente ao menos uma vez

9. Para refletir

- Em uma rotina real de auditoria da SEFAZ-PB, quais tarefas ganhariam mais em velocidade usando os assistentes visuais, e quais exigem que o script T-SQL fique documentado e versionado?
- O painel **Mostrar Script** do editor de dados resolve completamente a preocupação de rastreabilidade, ou ainda haveria vantagem em manter os **UPDATEs** de triagem como scripts formais, como os do exercício integrador?
- Que riscos existem em depender só da inferência automática de tipos do assistente de importação, sem revisar manualmente o esquema sugerido — especialmente diante de um arquivo com erros propositais, como o do exercício integrador?

10. Referências

Todas as capturas de tela e descrições técnicas deste material foram retiradas da documentação oficial da Microsoft (Microsoft Learn), atualizada em 2025-2026:

1. Microsoft Learn — *Overview - MSSQL Extension for Visual Studio Code*. Disponível em: <https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/mssql/mssql-extension-visual-studio-code?view=sql-server-ver17>
2. Microsoft Learn — *Database Operations - MSSQL Extension for Visual Studio Code* (Import Flat File). Disponível em: <https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/mssql/mssql-database-operations?view=sql-server-ver17>
3. Microsoft Learn — *Schema Designer - MSSQL Extension for Visual Studio Code*. Disponível em: <https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/mssql/mssql-schema-designer?view=sql-server-ver17>
4. Microsoft Learn — *Schema Designer with GitHub Copilot*. Disponível em: <https://learn.microsoft.com/en-us/sql/tools/visual-studio-code-extensions/mssql/mssql-schema-designer-copilot?view=sql-server-ver17>
5. Azure SQL Dev Corner (Microsoft DevBlogs) — *MSSQL Extension for VS Code: Introducing Schema Designer (Preview)*. Disponível em: <https://devblogs.microsoft.com/azure-sql/vs-code-mssql-schema-designer/>

6. Azure SQL Dev Corner (Microsoft DevBlogs) — *MSSQL Extension for VS Code: SQL Notebooks, AI-Powered Schema Design, Data API builder & More*. Disponível em: <https://devblogs.microsoft.com/azure-sql/vscode-mssql-march-2026/>
7. Microsoft — *What's Happening with Azure Data Studio*. Disponível em: <https://learn.microsoft.com/en-us/sql/tools/whats-happening-azure-data-studio?view=sql-server-ver17>

Nota sobre as imagens: todas as capturas de tela reproduzidas neste documento são hotlinks para os arquivos de imagem hospedados oficialmente pela Microsoft em learn.microsoft.com, extraídos diretamente dos tutoriais públicos indicados acima — não foram usadas capturas de tela de terceiros ou de fontes não oficiais.